Interview Preparation Guide

Narrative Engine — ML/AI Engineer Assignment

Part 1: Topic Evolution Pipeline | Part 2: Semantic Topic Search

EASY Conceptual / Definitions	**MEDIUM** Applied / Design Decisions
HARD Trade-offs / System Design	**FOLLOW-UP** Interviewer Challenges

PART 1 — Topic Evolution Pipeline

1. UNDERSTANDING & FUNDAMENTALS

Q1: **[EASY]** What is the Jaccard similarity score and why did you use it as the primary metric for comparing topics?

Ideal Answer:

Jaccard similarity measures the overlap between two sets — specifically, the intersection divided by the union of those sets.

I used it as the primary metric because the core problem is: "which posts appear in both topics?" Each topic is essentially a set of post IDs, and Jaccard directly quantifies how much the same real-world content is being grouped together. It is:

- Symmetric — topic A vs B equals topic B vs A
- Naturally bounded between 0 and 1
- Independent of topic size — a small stable topic with 10 posts scores correctly against a large one
- Interpretable — Jaccard 0.8 literally means 80% of content is shared

The key insight is that post IDs are ground truth: the same post re-appearing in two runs is unambiguous evidence that two topics are related, regardless of how titles or summaries are phrased.

Follow-up: If two topics have a Jaccard score of 0.05 but both discuss the exact same event — how would you detect that?

Q2: **[EASY]** Explain what TF-IDF is and why you applied it here on top of Jaccard similarity.

Ideal Answer:

TF-IDF (Term Frequency–Inverse Document Frequency) is a text vectorization technique that weights words by how frequently they appear in a document relative to how common they are across all documents. Rare, discriminating words get high scores; common stopwords get near zero.

In this solution, TF-IDF + cosine similarity is used as a secondary signal alongside Jaccard. The reason is that Jaccard alone is post-overlap-based — it does not capture semantic similarity in the topic titles or summaries. TF-IDF helps in cases like:

- Two topics that share few overlapping posts but discuss the same subject
- Topics where the NE model re-assigned posts slightly differently between runs

Together they form a multi-signal approach: Jaccard gives content-level signal, TF-IDF+cosine gives metadata-level signal. Both are recorded in the output, giving analysts two lenses on similarity.

Follow-up: Why not use TF-IDF as the primary signal instead of Jaccard?

Q3: **[EASY]** What is cosine similarity and how does it differ from Jaccard in this context?

Ideal Answer:

Cosine similarity measures the angle between two vectors in a high-dimensional space. Two documents are similar if their TF-IDF vectors point in the same direction, regardless of their magnitude (length).

Key differences in this context:

- Jaccard operates on sets of post IDs — it measures literal content overlap
- Cosine similarity operates on text vectors (titles + summaries) — it measures semantic/linguistic similarity
- Jaccard is deterministic given the post assignments; cosine varies with vocabulary choices in the AI-generated titles/summaries

Cosine similarity is more susceptible to noise from AI-generated text (synonyms, phrasing variation), which is why Jaccard is the primary metric for categorization.

Q4: **[MEDIUM]** Walk me through your categorize_evolution() function. Why did you choose these specific Jaccard thresholds?

Ideal Answer:

The function maps a Jaccard score to a human-readable label for stakeholders:

- Stable (≥ 0.8): The vast majority of posts are shared. The topic is essentially unchanged — perhaps minor additions or edge post re-assignments.
- Evolved (0.5 – 0.8): A clear majority of posts overlap, but there has been meaningful change — new angles added or older content dropped. This is the "same story, new chapter" category.
- Fragmented (0.1 – 0.5): Some posts are shared but not the majority. The topic may be splitting, merging, or diverging significantly.
- Dropped/Noise (< 0.1): Almost no post overlap. This pair is likely noise from the pipeline generating spurious connections, or the topic has genuinely died.

The thresholds are heuristics — they are reasonable starting points based on general NLP practice but should be validated against ground truth labels. The right thresholds are domain-specific.

Follow-up: What would happen to your categorization if the Narrative Engine ran on dramatically different-sized windows?

2. DESIGN DECISIONS

Q5: **[MEDIUM]** Why did you choose post-overlap (Jaccard on post IDs) as your primary matching signal rather than purely comparing topic titles or summaries?

Ideal Answer:

This is the central design insight: post IDs are ground truth, AI-generated text is not.

Three reasons:

- Titles and summaries are generated by the Narrative Engine — they may use different phrasings, synonyms, or levels of detail across runs even for identical topics.
- Post IDs are deterministic. If the same post ID appears in two topics across two runs, that is an objective signal, not a model-dependent artifact.
- This also respects the constraint that we cannot use an LLM to compare topics — text-based LLM comparison would be higher quality but is explicitly disallowed.

The combination of both signals (Jaccard + cosine) provides a richer picture, but the post-overlap signal drives the primary categorization decision.

Follow-up: What if the data pipeline had an ID generation bug and some posts had duplicate IDs across runs?

Q6: **[HARD]** How does your pipeline detect a topic SPLIT (one topic becoming two) and a MERGE (two topics becoming one)? Is it explicit or implicit?

Ideal Answer:

It is implicit, not explicit — which is an important limitation to acknowledge.

The run_comparison method generates ALL pairs of (prior_topic, later_topic) that share at least one post. This means:

- Split detection: If Topic A in Run 1 maps to BOTH Topic B and Topic C in Run 2 with moderate Jaccard scores (e.g. 0.3 and 0.4), the output will contain two rows for Topic A — this is the signature of a split.
- Merge detection: If Topic B and Topic C in Run 1 both map to Topic A in Run 2, that is a merge signature — Topic A appears twice as the "New Topic" column.

The output DataFrame does not have a dedicated "Split" or "Merge" column — analysts would need to detect these patterns post-hoc by grouping on Earlier Topic or New Topic and looking for one-to-many mappings.

An explicit improvement would be to add a post-processing step that: (1) identifies topics with multiple high-overlap partners, (2) computes the partition of posts, (3) labels rows explicitly as SPLIT or MERGE.

Follow-up: How would you handle the case where a topic splits and one half merges with something else simultaneously?

Q7: **[MEDIUM]** You merge three post DataFrames on post ID with an outer join, then filter for posts appearing in at least 2 runs. What is the reasoning behind this design?

Ideal Answer:

The outer join is essential because the three runs use overlapping but not identical date windows. Not every post will appear in all three runs:

- A post from Jan 20 may appear in Run 1 and Run 2 but not Run 3 (which starts Jan 22).
- Dropping posts that do not appear in all three runs would create a massive false negative problem — we would miss valid topic continuations.

The ≥2 filter serves a specific purpose: we only care about posts that create a bridge between at least two runs. A post appearing in only one run cannot inform any between-run comparison — it would only add noise to the similarity computation.

The fillna(-1) followed by replace(-1, "-1") ensures that missing topic assignments are handled as valid string values and do not create NaN-based join artifacts.

Follow-up: What happens to your metrics if a post appears in Run 1 and Run 3 but NOT Run 2?

3. ALTERNATIVE APPROACHES

Q8: **[HARD]** What alternative approaches could you have used to compare topics, and why did you not choose them?

Ideal Answer:

BERTopic / Sentence Embeddings on posts: You could embed all posts and compare topic centroids across runs. More semantically rich, but significantly more compute-intensive, harder to

interpret, and potentially introduces drift in embedding space. Also harder to justify "same topic" claims to stakeholders.

Exact entity matching: Compare topics purely by key_entities. Fast and interpretable, but brittle — the NER model may extract different surface forms or miss entities across runs, causing false negatives.

Graph-based topic tracking: Build a bipartite graph of (run1_topics) — (run2_topics) connected by Jaccard weight, then apply community detection. More principled for multi-topic merges/splits, but adds dependency on graph libraries and complexity.

LDA/NMF re-alignment: Run a global topic model across all three runs and re-align local topics to global ones. Theoretically principled but violates the independence constraint of the runs and adds complexity.

The chosen approach (Jaccard on post IDs + TF-IDF cosine) is the best balance of interpretability, correctness, and computational efficiency for this specific data structure.

🔁 **Follow-up:** In what scenario would the graph-based approach outperform yours significantly?

Q9: **[MEDIUM]** Could you have used Overlap Coefficient instead of Jaccard? When would it be better?

✅ **Ideal Answer:**

Yes — Overlap Coefficient = |A ∩ B| / min(|A|, |B|). It measures overlap relative to the SMALLER set, making it insensitive to size asymmetry.

When it would be better: When one topic is a strict subset of another — for example, a large "AI News" topic in Run 1 that splits into "AI Regulation" and "AI Research" in Run 2. Jaccard would penalize the smaller child topics because the union is large. Overlap Coefficient would correctly show high similarity between parent and each child.

The trade-off: Overlap Coefficient can be misleadingly high when a tiny topic happens to be entirely contained in a large one, even if the large topic has moved on completely. Jaccard is more conservative and penalizes this, which is the correct behavior for detecting stable topics.

4. EDGE CASES & FAILURE SCENARIOS

Q10: **[MEDIUM]** How did you handle the topic -1 (outlier) posts in your pipeline? What edge cases does this create?

✅ **Ideal Answer:**

Topic -1 is stored as the string "-1" after the int conversion and fillna logic. This means all outlier posts are grouped under a single "super-topic" labeled "-1" in the comparison.

The critical edge case: If many posts are outliers in Run 1 but the same posts get assigned to real topics in Run 2, the pipeline will compute a Jaccard score between topic -1 (Run 1) and real topics (Run 2). This would make topic -1 appear to have many "successors", which is semantically meaningless.

Better handling: Filter out all rows where run_1, run_2, or run_3 == "-1" before running comparisons. Topic -1 should be treated as a null/unknown category, not a real topic. The current code does not do this filtering explicitly, which is a bug.

🔁 **Follow-up:** How many posts in the dataset are outliers, and what percentage does that represent? How would you surface this to stakeholders?

Q11: **[HARD]** Your pipeline processes only posts that appear in at least 2 runs. What topics or changes might you be systematically missing?

Ideal Answer:

Several categories of topics will be invisible to the pipeline:

- New topics that emerge in Run 3 only: If all posts in a Run 3 topic are new posts (post-dating Run 1 and Run 2), none of those posts appear in the merged DataFrame, so the topic has no representation.
- Dying topics: A topic in Run 1 whose posts do not recur in Run 2 or 3 (perhaps it was a short news spike) will be invisible. It looks like it was never there.
- Topics from posts near date boundaries: Posts at the edge of the overlap window are more likely to appear in only one run.

To fix this: After the main pipeline, add a separate pass that identifies topics in each run that have zero matches in the merged dataset, and label them as "Emerging" (new topics) or "Expired" (topics that disappeared).

Follow-up: How would you detect if a topic completely disappeared between Run 2 and Run 3?

Q12: **[MEDIUM]** What happens to your pipeline if the Narrative Engine produces a very large "catch-all" topic containing thousands of posts?

Ideal Answer:

A large catch-all topic (e.g., a general "Politics" topic with 3,000 posts) would cause Jaccard inflation issues:

- It will match with nearly every other topic in the other runs because its post set is so large that intersections are common.
- But each individual Jaccard score will be low because the union is dominated by the large topic's massive post set.
- This creates many low-Jaccard "Fragmented" pairings in the output that are not meaningful.

Mitigation strategies:

- Filter out topics above a size threshold before comparison (or flag them separately).
- Use a quality score (e.g., topic coherence from the NE) to down-weight or exclude poorly-performing generalist topics.
- Apply a minimum Jaccard threshold before including a pair in results (the current code does not have a minimum threshold beyond intersection > 0).

5. OPTIMIZATION & SCALABILITY

Q13: **[HARD]** What is the time complexity of your run_comparison method? How would it scale to millions of topics?

Ideal Answer:

The current complexity is $O(T_1 \times T_2 \times P)$ in the worst case, where T_1 and T_2 are topic counts per run and P is the average posts per topic.

The inner loop iterates over all pairs of (prior_topic, later_topic). With 442 × 271 = ~120,000 pairs, this is manageable. But at millions of topics, this becomes infeasible.

Scalability improvements:

- Inverted index: Build a map of post_id → topic_id. For each prior topic, look up only the later topics that share at least one post. This reduces the effective pairs dramatically.
- MinHash / LSH: Approximate Jaccard similarity at scale using MinHash and Locality-Sensitive Hashing. Reduces to O(T × b × r) where b×r are band and row parameters.
- Spark / distributed computing: Shard topics by post ID hash ranges and compute similarities in parallel.
- Threshold early exit: Skip pairs where intersection is very small relative to sizes (Jaccard upper bound check without computing full union).

🔁 **Follow-up:** What is the Jaccard upper bound trick and how would you implement it?

Q14: **[MEDIUM]** Your TF-IDF vectorizer is re-fit for every pair of topics in the inner loop. What is the problem with this and how would you fix it?

✅ **Ideal Answer:**

This is a significant inefficiency bug. The vectorizer.fit_transform([text_p, text_l]) call inside the loop re-fits the vocabulary and computes IDF weights from scratch for every single topic pair. This means:

- IDF weights are computed from only 2 documents — this is statistically meaningless (IDF on 2 docs is essentially just binary).
- The vocabulary changes per pair, making cosine scores non-comparable across pairs.
- Performance: $O(T_1 \times T_2)$ vectorizer fits instead of O(1).

The correct approach:

- Fit the vectorizer ONCE on the entire corpus of all topic texts from both runs.
- Transform all topic texts in a batch to get a full matrix.
- Compute the full cosine similarity matrix at once using sklearn's cosine_similarity on the full matrices.

This reduces from $O(T^2)$ vectorizer fits to 1 fit and 1 matrix multiplication.

🔁 **Follow-up:** After fixing this, what would the correct interpretation of cosine similarity be that you could not claim before?

6. CODE-LEVEL DEEP DIVE

Q15: **[MEDIUM]** In get_entity_changes(), you extract only entity_list[0] from each entity cluster. What is the assumption here and when could it be wrong?

✅ **Ideal Answer:**

The data format for entities is a list of lists: [[all variations of entity 1], [all variations of entity 2], ...]. Index [0] is assumed to be the canonical/primary variant of each entity.

This assumption could be wrong when:

- The NER model stores variants in arbitrary order — e.g., ["Trump", "Donald Trump", "President Trump"] vs ["Donald Trump", "Trump"] across runs. The canonical form differs, causing "Trump" and "Donald Trump" to appear as separate entities in the diff even though they are the same.
- The entity cluster is empty (edge case not handled) — entity_list[0] would raise an IndexError.

Better approach: Normalize entities by choosing the longest or most frequent variant as canonical, or compare entire sets of variants across all entity clusters (set intersection on all variants, not just index 0).

Q16: **[MEDIUM]** In extract_data(), you do str(int(post["topic"])). What edge cases could cause this to fail or produce incorrect results?

Ideal Answer:

Several edge cases:

- If topic is a float like 3.0, int(3.0) = 3, which is fine. But if it is NaN (from a missing field), int(NaN) throws a ValueError.
- If topic is a string like "3a" or an empty string, int() will throw a ValueError.
- If topic is None (missing key + .get() default), the code will throw a TypeError since int(None) is invalid.
- The data description explicitly says "This is messy real world data with many edge cases" — so these failures are likely.

A safer pattern: use a try/except or pd.to_numeric() with errors="coerce", followed by fillna(-1) to handle all non-numeric topic IDs gracefully.

Follow-up: How would you add data validation / schema checking at the point of ingestion?

7. VALIDATION STRATEGY

Q17: **[HARD]** How would you validate that your topic evolution pipeline is producing correct results? Walk me through a full validation strategy.

Ideal Answer:

A rigorous validation strategy would have three layers:

Layer 1 — Human annotation (gold standard): Manually annotate a sample of ~100–200 topic pairs as "same", "evolved", "split", "merged", or "new/dead". Compute precision, recall, and F1 for each category against pipeline output.

Layer 2 — Threshold sensitivity analysis: Run the pipeline with varying Jaccard thresholds (0.5, 0.6, 0.7, 0.8, 0.9) and measure how category distributions change. Stable distributions across a range suggest robust thresholds.

Layer 3 — Sanity checks on known events: The data covers news from late January to late February 2026. Identify 2–3 major news events known to have evolved (e.g., a story that escalated). Verify the pipeline correctly marks those as "Evolved" or "Fragmented".

Automated regression tests: Create synthetic datasets with known ground truth (3 posts per topic, guaranteed overlap patterns) and assert pipeline output matches expected labels exactly.

Follow-up: What metrics would you use to measure the quality of the categorization beyond accuracy?

PART 2 — Semantic Topic Search

1. UNDERSTANDING & FUNDAMENTALS

Q18: **[EASY]** What is a sentence transformer and why did you choose multi-qa-MiniLM-L6-cos-v1 specifically over all-MiniLM-L6-v2?

Ideal Answer:

A sentence transformer is a BERT-variant fine-tuned with a siamese/triplet network objective to produce semantically meaningful sentence-level embeddings — i.e., sentences with similar meaning have similar vectors in the embedding space.

all-MiniLM-L6-v2 is a general-purpose sentence similarity model. multi-qa-MiniLM-L6-cos-v1 is specifically fine-tuned for asymmetric retrieval tasks: encoding a query against a set of passages/documents.

This distinction is crucial here: the user's query is structurally very different from the topic "documents" (which are title+summary+entities). The multi-qa model is optimized for this asymmetric scenario — it produces better query-document matching than a symmetric model.

The comment in the code (showing both options) suggests this was a deliberate, tested decision — a good demonstration of model selection awareness.

Follow-up: How would you evaluate whether this model selection was actually correct for this use case?

Q19: **[EASY]** Explain what the deduplication step does in compare_topics() and why it is necessary before embedding.

Ideal Answer:

The deduplication step uses Jaccard similarity on token sets to identify topics that are likely duplicates across the three runs — the same real-world topic appearing in multiple files.

Why it is necessary:

- Without deduplication, the same topic would appear 2–3 times in the embedding index (once per run). A search would return near-duplicate results occupying multiple top-5 slots, providing the user with less diversity of information.
- It reduces the embedding index size, improving search latency and memory usage.
- It ensures that a single query does not surface the "Groundhog Day" topic from all three runs simultaneously — the user gets unique topics.

The Jaccard threshold of 0.5 is a design choice — lower values deduplicate more aggressively, potentially merging distinct but related topics.

Follow-up: What would a Jaccard threshold of 0.1 do to your search results? What about 0.95?

2. DESIGN DECISIONS

Q20: **[HARD]** You are not allowed to use an LLM to check every topic for similarity to the query. Why is embedding + cosine search a valid alternative? What does it sacrifice?

Ideal Answer:

Embedding + cosine search works because the sentence transformer creates a shared vector space where semantically similar texts have similar directions. A query about "Super Bowl political controversy" and a topic about "Kendrick Lamar's halftime show political protest" should have nearby vectors.

What it gains over LLM per-topic comparison:

- Speed: O(1) embedding of query + O(T) cosine operations vs O(T) LLM API calls. For millions of topics, this is the difference between milliseconds and hours.
- Cost: A single embedding call vs T inference calls at full LLM cost.

What it sacrifices:

- Reasoning: An LLM can understand nuanced user intent like "I want political controversy, NOT sports analysis." Embeddings encode both dimensions but cannot reason about which dimension is more important.
- Context: LLMs can follow multi-step reasoning ("show me topics that would be interesting to a researcher studying media bias"). Embeddings do not reason.
- Negation: "I do NOT want X" is poorly handled by cosine similarity — the embedding of "not violent" and "violent" can be close.

Follow-up: How would you handle a user query that contains negation — e.g., "I want protest topics but NOT violent ones"?

Q21: **[MEDIUM]** Your combination string is: "Title: {title}; Summary: {summary}; entities: {entities}". Why did you format it this way and what are the trade-offs of this design?

Ideal Answer:

The structured format serves two purposes:

- Prefix labels (Title:, Summary:, entities:) help the transformer understand which part of the text is which — though modern transformers largely ignore this without specific fine-tuning.
- Including all three signals (title, summary, entities) in one string ensures the embedding captures the full semantic profile of the topic — not just its headline or just its named entities.

Trade-offs:

- Entities are extracted as primary variants only — if the topic is about "Donald Trump" but the entity list reads "Trump", a query for "Donald Trump" might miss it.
- Summaries can be noisy or hallucinated by the NE model — injecting low-quality text degrades the embedding.
- Long combination strings may exceed the model's context window (128 tokens for MiniLM), causing truncation and loss of information.
- Entity weighting: Entities are equally weighted with all other text. But for proper-noun-heavy queries, entities are more discriminating and should be up-weighted.

Follow-up: How would you handle topics with very long summaries that exceed the model token limit?

3. ALTERNATIVE APPROACHES

Q22: **[HARD]** What is BM25 and how does it compare to your semantic embedding approach for this search problem?

Ideal Answer:

BM25 (Best Match 25) is a probabilistic keyword-based ranking function — an evolution of TF-IDF that accounts for document length normalization and term saturation. It is the backbone of Elasticsearch and many production search systems.

BM25 vs Semantic Embeddings for this use case:

- BM25 requires keyword overlap — the user query must share words with the topic text. "Super Bowl halftime controversy" would miss a topic titled "Performers face political backlash at NFL championship event".
- Semantic embeddings handle paraphrase/synonym matching — "halftime show" and "halftime performance" map to similar vectors.
- BM25 is much faster and has no embedding overhead — ideal for very large corpora.
- BM25 handles exact entity names better (e.g., "Kendrick Lamar") where embeddings may lose specificity.

Best production approach: Hybrid BM25 + semantic embedding (Reciprocal Rank Fusion or cross-encoder re-ranking). BM25 for recall, embeddings for precision.

Follow-up: How would you implement a hybrid BM25 + embedding system? What would the architecture look like?

Q23: **[MEDIUM]** Could you have used FAISS or another ANN library for the search instead of brute-force cosine similarity? When would that matter?

Ideal Answer:

Yes. The current implementation computes exact cosine similarity between the query vector and ALL topic embeddings — O(T × d) where d is embedding dimension (384 for MiniLM).

For this problem (hundreds to low thousands of topics), brute-force is perfectly fine — it runs in microseconds. FAISS would provide no practical benefit at this scale.

FAISS / ANN becomes necessary when:

- T > 100,000 topics — brute-force becomes too slow for interactive search (> 100ms latency).
- Running millions of queries per day — even O(T) becomes significant under high QPS.

FAISS (HNSW or IVF index) trades a small accuracy loss (approximate nearest neighbors vs exact) for 10–1000x speedup. For a search application where top-5 relevance is sufficient, this is an acceptable trade-off.

4. EDGE CASES & FAILURE SCENARIOS

Q24: **[HARD]** A user submits the query: "I want to see topics about the recent violence during protests." How might your pipeline fail or produce poor results?

Ideal Answer:

Several failure modes for this specific query:

- Dual topic sensitivity: "violence" and "protests" are both heavy anchors in the embedding. If there are topics about protests without violence, or violence without protests, they may score almost as high as truly matching topics.
- Recency blindness: The query says "recent" — but all three runs are from the same February 2026 window. The system has no ability to prioritize more recent topics over older ones within the dataset.

- Emotional register: "I am interested in seeing what emotions are expressed" — this intent is invisible to keyword-based and pure semantic embedding approaches. The embedding of the combination string does not capture sentiment or emotional register.
- Missing context: If the protest topics are only described with neutral entity names (e.g., "Gaza protests") and no violence-specific vocabulary, the cosine similarity may miss them entirely.

Improvement: Use an LLM to expand the query into a richer semantic document before embedding (HyDE — Hypothetical Document Embeddings).

🔄 **Follow-up:** What is HyDE and how would you implement it as an enhancement here?

Q25: **[MEDIUM]** Your deduplication uses a Jaccard threshold of 0.5. What happens if two genuinely different topics happen to share 50%+ of their vocabulary?

✅ **Ideal Answer:**

This creates a false duplicate — a unique topic gets dropped from the embedding index. It would then be permanently invisible to any search query.

When this is likely to occur:

- Short topics with small token sets — even 3–4 shared words can push Jaccard above 0.5 for short texts.
- Topics that share a common theme but diverge in focus — e.g., "Super Bowl TV ratings" vs "Super Bowl halftime TV broadcast controversy" share many tokens.

Improvements:

- Use TF-IDF weighted Jaccard (or WMD) instead of raw token Jaccard — this down-weights common/generic words.
- Apply deduplication at the embedding level instead — two topics are duplicates if their sentence embeddings have cosine similarity > 0.95 (much stricter semantic identity test).
- Instead of dropping duplicates, merge them by averaging their embeddings — preserves information while reducing redundancy.

5. OPTIMIZATION & SCALABILITY

Q26: **[HARD]** How would you productionize this search system to serve real-time queries at scale? Walk me through the architecture.

✅ **Ideal Answer:**

A production architecture would separate the offline indexing pipeline from the online serving layer:

Offline pipeline: New run outputs trigger a batch process that: (1) deduplicates new topics against the existing index, (2) generates embeddings in batch on GPU, (3) upserts vectors into a vector database (Pinecone, Weaviate, or pgvector).

Online serving: User query → LLM query expansion (optional HyDE) → embed query → ANN lookup in vector DB → return top-K results with metadata. Target latency: < 200ms.

Monitoring: Log query embeddings, result IDs, and user click-through rates. Use click-through as implicit relevance signal. Monitor embedding drift — if the sentence transformer is updated, re-embed the full index.

Caching: Cache query embeddings for repeated queries (Redis with TTL). Pre-compute embeddings for known common query patterns.

A/B testing: Run experiments with different models (BM25 vs embedding vs hybrid) using interleaved ranking or shadow mode evaluation.

🔁 **Follow-up:** How would you handle a situation where the embedding model needs to be swapped out for a better one? What is the migration path?

6. REAL-WORLD & CROSS-CUTTING QUESTIONS

Q27: **[HARD]** Across both parts, you had to process "messy real-world data." How did you approach data quality and what systematic problems might remain?

✅ **Ideal Answer:**

Handled defensively in code:

- str(int(x)) pattern converts floats/ints to strings uniformly.
- isinstance() checks on summary (list vs string) handle format inconsistencies.
- fillna(-1) / replace(-1, "-1") handles missing join values.
- Topic -1 filtering for outliers.

Likely remaining issues:

- Malformed JSON — no validation at load time; a truncated file would crash the pipeline silently after partial processing.
- Unicode/encoding issues in post text — social media data is rich in emojis, non-ASCII characters, URLs, which can break TF-IDF tokenization.
- Duplicate post IDs — the problem statement notes IDs are random hashes. Hash collisions or bugs could create false cross-topic bridges.
- None entities — the problem statement explicitly mentions some outlier posts have None entities. Code accesses entities without guarding against None in all paths.

🔁 **Follow-up:** What is the first data quality check you would add before any processing begins?

Q28: **[MEDIUM]** If a stakeholder asked you to explain WHY two topics were matched as "Evolved," what information would you surface from your pipeline?

✅ **Ideal Answer:**

The pipeline already provides the right building blocks for explainability:

- Jaccard Score: "X% of the posts discussing this topic in Run 1 reappeared in the Run 2 topic."
- Cosine Similarity: "The titles and summaries of both topics are Y% textually similar."
- Added Entities: "New key entities in the evolved topic: [list]."
- Removed Entities: "Entities that were important but are no longer prominent: [list]."
- Earlier Title vs New Title: Side-by-side comparison of how the narrative framing changed.

For a richer explanation, you could also surface: (1) the posts that are new to the later topic (not in the prior), (2) the posts that dropped out, and (3) a sample of the actual text for each group. This gives stakeholders a story, not just numbers.

Q29: **[HARD]** Looking across both solutions holistically — what is the single biggest weakness, and what would your next iteration look like?

✅ **Ideal Answer:**

The biggest weakness is the lack of an end-to-end quality signal. Neither part has a mechanism to measure whether the outputs are actually correct — both rely entirely on heuristics that were never validated against ground truth.

Part 1's Jaccard thresholds are arbitrary. Part 2's search results are never evaluated for relevance. Without ground truth, we cannot know if these systems are working or just producing plausible-looking output.

Next iteration priorities:

- Part 1: Build a small annotation tool to label 200 topic pairs. Use these to calibrate Jaccard thresholds, fix the topic -1 handling, and measure F1 per category. Add explicit SPLIT/MERGE detection.
- Part 2: Run 20–30 example queries with manual relevance judgements. Compute nDCG@5. Test HyDE for query expansion. Implement hybrid BM25+embedding. Fix the TF-IDF vectorizer bug in Part 1 as a side effect of shared refactoring.
- Both: Add a data validation layer at ingestion, structured logging for observability, and unit tests for edge cases.

🔁 **Follow-up:** If you had one week to productionize just one of the two parts, which would you choose and why?

— End of Interview Preparation Guide —